

SOLUCIONARIO DEL EXAMEN PARCIAL
CICLO 2020 - II
ARQUITECTURA DE COMPUTADORES (CC221)

Por: César Martín Cruz Salazar

1. (4 pts.) Calcule los valores de los registros TH0 y TL0 en Modo 0 y en Modo 1 para un reloj de sistema de 11.0592Mhz y para una frecuencia de 888hz.

Sol.

Se hace el cálculo para generar una frecuencia de 888hz. Para ello tengo un periodo de $1/888=0.0011261261261261$. Se aplica la relación para el semiperiodo 0.000563 por tanto:

En Modo 0

$$(8192-j) \times (12/11.0592 \times 10^6) s = 0.000563s$$

j=7673 en modo 0 → **TH0=1Dh, TL0=F9h**

En Modo 1

$$(65536-j) \times (12/11.0592 \times 10^6) s = 0.000563s$$

j=65017 en modo 1 → **TH0=FDh, TL0=F9h**

2. (2 pts.) ¿Escriba el código que se necesita para habilitar las interrupciones externas 0 y 1 y la interrupción interna del timer 1 ?.

Sol.

setb IT0

setb IT1

setb ET1

setb EX0

setb EX1

setb P3.2

setb P3.3

setb EA

setb TR1

3. (4 pts.) Describa las características del MSP430G2553 de TI y detalle 3 modos de direccionamiento con ejemplos. Mencione al menos dos diferencias con el 8051.

Sol.

Características:

Microcontrolador de 16 bits

Flash ROM de 16kbytes

Contiene un convertidor A/D de 10 bits y sensor de temperatura

Memoria RAM de 512 bytes

Modos de Direccionamiento:

1)Modo Registro: Mueve el contenido de un registro a otro registro.

Ejm. Mov R5,R7

2)Modo Inmediato: Mueve un valor constante a un registro.

Ejm. Mov #0F5ACh,R11

3)Modo Indirecto: Mueve el valor contenido en la dirección contenida en un registro a otro registro.

Ejm. Mov @R14,R10

Dos diferencias con el 8051.

El 8051 es de 8 btis, el MSP430G2553 es de 16 bits.

Arquitectura del 8051 es CISC, el de MSP430G2553 es RISC.

4. (3 pts.) Desarrolle un programa que calcule el **décimo** y el **doceavo** término de la serie de Fibonacci (1,1,2,3,5,8,13,21,34,55,89,144). Enviar ambos resultados al terminal.

Sol.

```
;décimo y doceavo valor de Fibonacci
;UPDATE FREQ: 1000
    org 0h
    lcall init
    lcall print
    db "decimo y doceavo valor de"
    db 0dh,0ah
    db " la serie de Fibonacci "
    db 3Ah
    db 0dh,0ah,0
    ;a,b y c=a+b
    mov R3,#10
    mov R4,#2
    mov R0,#1      ;R0 hace las veces de a
    mov R1,#1      ;R1 hace las veces de b
lazo:
    mov A,R0
    add A,R1
    mov R2,A      ;R2 hace las veces de c
    inc R4
    cjne R4,#10,sigue
    mov A,R2
    lcall dos_digitos_decimales
    mov A,30h
    lcall sndchr
    mov A,31h
    lcall sndchr
    mov A,#','
    lcall sndchr
    mov A,#20h
    lcall sndchr
sigue:
    cjne R4,#12,sigue_
    mov A,R2
    lcall tres_digitos_decimales
    mov A,30h
    lcall sndchr
    mov A,31h
    lcall sndchr
    mov A,32h
    lcall sndchr
sigue_:
    mov A,R1
    mov R0,A      ;a=b
    mov A,R2
    mov R1,A      ;b=c
    djnz R3,lazo
    sjmp $
```

```

;=====
;Devuelve tres codigos ASCII de tres digitos decimales
;=====
tres_digitos_decimales:
    mov B,#100
    div AB
    add A,#30h
    mov 30h,A
    mov A,B
    mov B,#10
    div AB
    add A,#30h
    mov 31h,A
    mov A,B
    add A,#30h
    mov 32h,A
    ret

;=====
;Devuelve dos codigos ASCII de dos digitos decimales
;=====
dos_digitos_decimales:
    mov B,#10
    div AB
    add A,#30h
    mov 30h,A
    mov A,B
    add A,#30h
    mov 31h,A
    ret

sndchr:
    clr TI
    mov SBUF,A

txloop:
    jnb TI,txloop
    ret

init:
    mov PCON,#80H
    mov TMOD,#20H
    mov TCON,#41H
    mov TH1,#0FDH
    mov SCON,#50H
    ret

;=====
;Subrutina print
;toma la cadena inmediatamente que sigue a "lcall"
;y lo envia por el puerto serie. La cadena debe terminar
;con un nulo(0).
;Esta subrutina retornará a la instrucción que sigue
;inmediatamente a la cadena.
;=====
print: pop dph ;coloca dirección de retorno en dptr
    pop dpl
prtstr:
    clr A
    movc A,@A+DPTR
    cjne A,#0h,mchrok
    sjmp prtdone
mchrok:
    lcall sndchr
    inc DPTR

```

```

    sjmp PRTSTR
prtdone:
    mov A,#1h
    jmp @A+DPTR
end

```

5. (7 pts.) Desarrolle un programa de Conversión de un número decimal de 8 bits a cualquier base (binario, octal, hexadecimal). Enviar el resultado al terminal. Ingresar el número de la base desde el teclado mediante un menú.

01)Base 2

02)Base 8

03)Base 16

Seleccione uno de los tres:

Indicar el UPDATE FREQ utilizado.

Si desea puede aplicar el siguiente algoritmo o hacerlo como a Ud. le parezca:

INICIO

Enteros Num, base, DirMem

DirMem $\leftarrow$ 37h

Mientras (Num es distinto a 0)

```

{
    DirMem  $\leftarrow$  Resto(Num/base)      ;Almaceno el resto de la división en la
                                       ;dirección de memoria(DirMem)

    Num  $\leftarrow$  Num/base                ; El nuevo "Num" es el cociente de "Num" entre la "base"
    DirMem  $\leftarrow$  DirMem-1             ;Decremento DirMem
}

```

FIN

Sol.

```

;UPDATE FREQ: 600
    org 0h
    lcall init
    lcall print
    db "01)Base 2"
    db 0dh,0ah
    db "02)Base 8"
    db 0dh,0ah
    db "03)Base 16"
    db 0dh,0ah
    db "Seleccione una opcion "
    db 3Ah,0
    lcall getbyt
    cjne A,#1,oct
    mov R6,#2
    lcall print
    db 0dh,0ah
    db "En binario "
    db 3Ah,0
    sjmp continua

oct:
    cjne A,#2,hexa
    mov R6,#8
    lcall print
    db 0dh,0ah
    db "En Octal "
    db 3Ah,0

```

```

                                sjmp continua
hexa:
                                mov R6,#16
continua:
                                mov R0,#37h
                                mov R3,#200    ;Es el número decimal a convertir
sigue:
                                mov A,R3
                                mov B,R6
                                div AB
                                mov @R0,B
                                mov R3,A
                                dec R0
                                cjne R3,#0,sigue
                                cjne R6,#2,octal
                                mov R7,#8
                                inc R0
lazo:
                                mov A,@R0
                                add A,#30h
                                lcall sndchr
                                inc R0
                                djnz R7,lazo
                                sjmp $
octal:
                                cjne R6,#8,hexadecimal

                                mov R7,#3
                                inc R0
lazo1:
                                mov A,@R0
                                add A,#30h
                                lcall sndchr
                                inc R0
                                djnz R7,lazo1
                                sjmp $
hexadecimal:
                                cjne R6,#16,termina
                                lcall print
                                db 0dh,0ah
                                db "En hexadecimal "
                                db 3Ah,0
                                mov R7,#2
                                inc R0
lazo2:
                                mov A,@R0
                                lcall prthex
                                inc R0
                                djnz R7,lazo2
termina:
                                sjmp $
sndchr:
                                clr TI
                                mov SBUF,A
txloop:
                                jnb TI,txloop
                                ret
init:
                                mov PCON,#80H
                                mov TMOD,#20H
                                mov TCON,#41H
                                mov TH1,#0FDH

```

```

        mov SCON,#50H
        ret
;=====
;Subrutina prthex
;Esta rutina toma el contenido del acumulador y lo envía
; vía serie como 2 dígitos ascii hexadecimal.
;=====
prthex:
    lcall binasc
    mov A,R2
    lcall sndchr
    ret
;=====
;Subrutina binasc
;Toma el contenido del acumulador y
;lo convierte en dos números ascii hexadecimales.
;El resultado es retornado en el acumulador y R2
;=====
binasc:
    mov R2,A ;se mueve a R2 para salvar el valor de A
    anl A,#0Fh ;se convierte el dígito menos significativo
    add A,#0F6h ;lo ajusta
    jnc noadj1
    add A,#07h
noadj1:
    add A,#3Ah ;lo hace ascii
    xch A,R2 ;pone el resultado en R2
    swap A ;se convierte ahora el dígito más significativo
    anl A,#0Fh
    add A,#0F6h ;lo ajusta
    jnc noadj2
    add A,#07h
noadj2:
    add A,#3Ah ;lo hace ascii
    ret
;=====
;Subrutina print
;toma la cadena inmediatamente que sigue a "lcall"
;y lo envía por el puerto serie. La cadena debe terminar
;con un nulo(0).
;Esta subrutina retornará a la instrucción que sigue
;inmediatamente a la cadena.
;=====
print: pop dph ;coloca dirección de retorno en dptr
    pop dpl
prtstr:
    clr A
    movc A,@A+DPTR
    cjne A,#0h,mchrok
    sjmp prtdone
mchrok:
    lcall sndchr
    inc DPTR
    sjmp PRTSTR
prtdone:
    mov A,#1h
    jmp @A+DPTR
;=====
; subrutina getbyt
; esta rutina lee un número ascii hexadecimal de 2 dígitos
; desde el puerto serie. El resultado es retornado en el acumulador
;=====

```

```

;
getbyt:
lcall getchr ; consigue el caracter ascii msb
lcall ascbn ; lo convierte a binario
swap A ; lo mueve a la mitad mas significativa del acumulador
mov B, A ; lo salva en B
lcall getchr ; consigue el caracter ascii lsb
lcall ascbn ; lo convierte a binario
orl A, B ; combina las dos mitades
ret
; =====
; subrutina ascbn
; esta rutina toma el caracter ascii pasado en el
; acumulador y lo convierte a un número binario de 4 bits
; que es retornado en el acumulador.
; =====
ascbn:
add a, #0d0h ; si chr < 30 entonces error
jnc notnum
clr c ; chequea si chr es 0-9
add a, #0f6h ; lo ajusta
jc hextry ; salta si chr no es 0-9
add a, #0ah ; si lo es entonces lo ajusta
ret
hextry:
clr acc.5 ;
clr c ; chequea si chr es a-f
add a, #0f9h ; lo ajusta
jnc notnum ; si no es a-f entonces error
clr c ; averigua si char es 46 o menos.
add a, #0fah ; ajusta el acumulador
jc notnum ; si carry=1 entonces no es un número hex
anl a, #0fh ; limpia bits no usados
ret
notnum:
; setb errorf ;
ret
getchr: jnb ri, getchr ; espera hasta que el caracter sea recibido
mov a, sbuf ; consigue el caracter
anl a, #7fh ; mask off 8th bit
clr ri ; limpia el bit de estado serial
ret
end

```